

Google Developer Group
Universitas Sriwijaya

Learning 15

Complex State with Zustand

Managing global client state



```
const filterByOrg = (study, filterByOrg) => {  
  if (filterByOrg === null || filterByOrg === '') {  
    return study;  
  }  
  return filterByOrg === study.lead_organization ? study : null;  
};  
  
const filterByStatus = (study, filterByStatus) => {  
  if (filterByStatus === null || filterByStatus === '') {  
    return study;  
  }  
  return filterByStatus === study.status ? study : null;  
};  
  
const filterStudies = (studies, filterByOrg, filterByStatus) => {  
  return studies.filter(study => {  
    return filterByOrg === null || filterByOrg === study.lead_organization &&  
      filterByStatus === null || filterByStatus === study.status;  
  });  
};
```

When State Becomes Complex

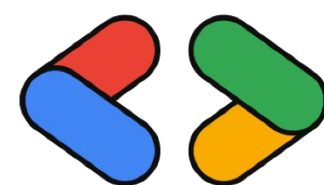
As applications grow, managing state becomes harder.

Example problems:

- many components need shared state
- passing props through many layers
- complex UI interactions

Example shared state:

- Theme
- User preferences
- Cart items
- Filters
- UI settings



Limitations of Context API

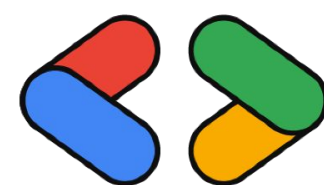
React Context works well for simple global state.

But for complex applications it can become difficult to manage.

Problems:

- frequent re-renders
- complex provider structure
- boilerplate code

Developers often use state management libraries.



What is Zustand?

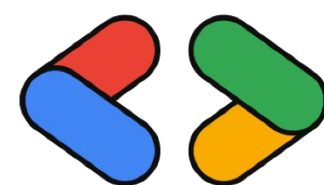
Zustand is a lightweight state management library for React.

But for complex applications it can become difficult to manage.

Features:

- simple API
- minimal boilerplate
- no provider required
- optimized re-rendering

Zustand is widely used in modern React applications.



Installing Zustand

Install Zustand:

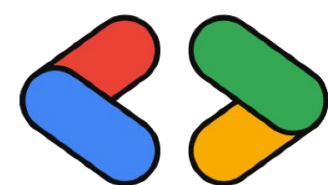
```
npm install zustand
```

Create a store:

```
import { create } from "zustand"

const useStore = create((set) => ({
  count: 0,
  increase: () => set((state) => ({
    count: state.count + 1
  })))
}))
```

A store holds state and actions.



Google Developer Group
Universitas Sriwijaya

Using Zustand in Components

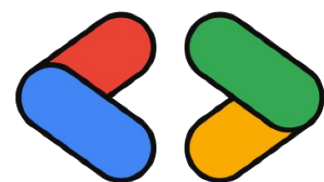
Example usage:

```
const count = useStore((state) => state.count)
const increase = useStore((state) => state.increase)
```

Example component:

```
<button onClick={increase}>
  Count: {count}
</button>
```

Components automatically update when state changes.



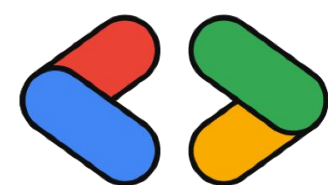
Example Use Case (Client State)

Example: Theme management.

```
const useThemeStore = create((set) => ({
  theme: "light",
  toggleTheme: () =>
    set((state) => ({
      theme: state.theme === "light"
        ? "dark"
        : "light"
    }))
}))
```

Component usage:

```
const theme = useThemeStore((state) => state.theme)
```



Client State vs Server State

Client State:

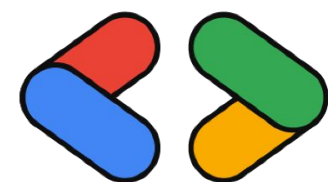
- UI state
- theme
- filters
- modal open

Server State:

- database data
- API responses
- Supabase data

Modern React Stack:

- Zustand → Client State
- TanStack Query → Server State



Integrating Zustand with Supabase Data

Zustand should not replace TanStack Query.

Instead, it complements it

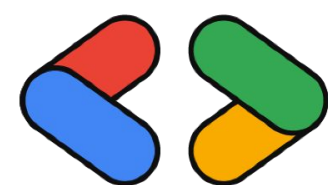
- TanStack Query → fetch students from Supabase
- Zustand → manage UI filters

Example store:

```
const useFilterStore = create((set) => ({
  search: "",
  setSearch: (value) => set({ search: value })
}))
```

Example store:

```
useQuery({
  queryKey: ["students", search],
  queryFn: fetchStudents
})
```





Thank You !